File: `src/components/ShellyDashboard.jsx`

Category: React Component (Main Feature / Orchestrator)
Runtime: Browser (React)
Primary role: top-level dashboard screen (coordinates data polling, interval selection, and chart panels)

Overview

`ShellyDashboard` is the main application feature component. It:

- defines available time intervals (UI selection)
- starts device polling via `useShellyStatus()`
- manages the "click outside closes popups" signal (`closeToken`)
- manages the chart panel system via `useChartPanels()`
- derives:
  - the latest sample
  - an anchor time (`latestMs`)
  - a filtered history window for charting
- renders:
  - interval selector
  - live readout card
  - min/max readout card
  - add-chart button
  - all chart panels

This component mainly coordinates other parts; it contains minimal business logic.

Imports

- React: `useMemo`, `useState`
- `useShellyStatus` from `../hooks/useShellyStatus`
- `LiveReadout` from `./LiveReadout.jsx`
- `MinMaxReadout` from `./MinMaxReadout.jsx`
- `useChartPanels` from `../hooks/useChartPanels`
- `ChartPanel` from `./ChartPanel.jsx`
- `chartPanelStyles` from `../styles/uiStyles`

Exported Component

`ShellyDashboard()`

Signature

```
ShellyDashboard() → JSX.Element
```

Parameters

- None.

Returns

- The full dashboard UI as JSX.

Internal Constants

`intervals`

An array of available time windows (milliseconds) for graphing and statistics, e.g.:

- 1 minute
- 3 minutes
- 5 minutes
- 15 minutes
- 30 minutes
- 45 minutes
- 1 hour

Each entry:

```
{ label: string, value: number }
```

Internal State

`selectedInterval`

```
useState<number>(intervals[1].value)
```

Represents the currently selected time window (in milliseconds).
Used for:

- filtering chart data
- computing time-axis display unit (`LineGraph` via `getTimeUnit`)
- computing interval min/max (`MinMaxReadout`)

`closeToken`

`useState<number>(0)`

A numeric "signal" used to tell all chart panels: **close popovers now**.

The parent increments the token when the user clicks outside chart panels.

`closeAllPopups()`

Increments token:

```
closeToken = closeToken + 1
```

This forces child `ChartPanel` components to detect a change and close their local popups.

Hooks Used

`dataHistory = useShellyStatus()`

Type

`ParsedDataPoint[]`

This starts polling automatically (every 2 seconds by default).

`actions = useChartPanels({ startEmpty: true, insertAtTop: true })`

Provides:

- `charts`
- `addChart`, `removeChart`, etc.
- helpers for dropdown and chart series building

Derived Values (`useMemo`)

`latestPoint`

```
useMemo(() => last element of dataHistory or null, [dataHistory])
```

Purpose:

- Provides current sample for `LiveReadout`
- Provides current values for `ChartPanel` info popover

`latestMs`

```
useMemo(() => latestPoint timestamp in ms or Date.now(), [latestPoint])
```

Purpose:

- Provides a stable "now" anchor for:
  - time window filtering
  - chart x-axis alignment
  - min/max interval window cutoff
- Uses device timestamp when available (preferred)

`filteredData`

```
useMemo(
  () => dataHistory.filter(dp.timestamp >= latestMs - selectedInterval),
  [dataHistory, latestMs, selectedInterval]
)
```

Purpose:

- Provides only the subset of historical data inside the selected interval window.
- Passed to every `ChartPanel` → `LineGraph`.

Dependency array explanation:

- When new data arrives → dataHistory changes → recompute
- When interval changes → selectedInterval changes → recompute
- When latest sample time shifts → latestMs changes → recompute

---

Rendering Structure

Root container

- Uses `onClick={closeAllPopups}`
- This means clicking anywhere in dashboard background closes all panel popups.

Interval selector

Dropdown controlled by `selectedInterval`.

Changing dropdown triggers:

- `setSelectedInterval(Number(e.target.value))`

Readout area

Renders side-by-side cards:

- `<LiveReadout data={latestPoint} />`
- `<MinMaxReadout dataHistory={dataHistory} selectedInterval={selectedInterval} nowMs={latestMs} />`

This container is:

- no wrapping (`flexWrap: "nowrap"`)
- horizontal scrolling enabled (`overflowX: "auto"`)

Add Chart button

Calls `actions.addChart()`.

Uses `stopPropagation` so the click does not trigger `closeAllPopups`.

Chart panel list

Maps `actions.charts` to `<ChartPanel />` components.

Each chart receives:

- the chart state object
- the filtered data window
- time selection
- latest point and timestamp
- closeToken signal

- actions object
- styling tokens

Side effects

None directly in this file.

All side effects come from:

- `useShellyStatus` (polling timer + network)
- `ChartPanel` local UI state
- `LineGraph` chart creation

Common edge cases

- If `dataHistory` is empty:
  - `latestPoint` becomes null
  - `LiveReadout` shows loading message
  - `MinMaxReadout` shows awaiting message
- If charts list is empty:
  - renders “Parameter Analysis Area” message